

1. Word Vectors

```
import gensim.downloader as api

model=api.load("word2vec-google-news-300")

print("Similar to king:")
for w,s in model.most_similar("king",topn=5):
    print(w,round(s,4))

res=model.most_similar(positive=['king','woman'],negative=['man'],topn=1)
print("\nking-man+woman =",res[0][0])

print("\nSimilarity:",round(model.similarity("king","queen"),4))

print("\nOdd one:",model.doesnt_match(["apple","banana","grape","car"]))
```

2. PCA Visualization

```
import gensim.downloader as api
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

model=api.load("word2vec-google-news-300")

words=["computer","algorithm","software","hardware","AI","cloud","database","network","security","data"]

vec=[model[w] for w in words]

pca=PCA(n_components=2).fit_transform(vec)

for i,w in enumerate(words):
    x,y=pca[i]
    plt.scatter(x,y)
    plt.text(x,y,w)

plt.show()

print("\nSimilar to AI:")
for w,s in model.most_similar("AI",topn=5):
    print(w,round(s,4))
```

3. Custom Word2Vec

```
from gensim.models import Word2Vec
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

corpus=["doctor treats patient","insulin for diabetes","heart diseases common","infection spreads fast","vaccines build immunity"]
```

```

data=[s.split() for s in corpus]

model=Word2Vec(data,size=50,min_count=1)

print("Similar to diabetes:")
for w,s in model.wv.most_similar("diabetes",topn=5):
    print(w,round(s,3))

words=list(model.wv.index2word)
vec=model.wv[words]

tsne=TSNE(n_components=2,perplexity=3).fit_transform(vec)

for i,w in enumerate(words):
    x,y=tsne[i]
    plt.scatter(x,y)
    plt.text(x,y,w)

plt.show()

```

4. Prompt Enrichment

```

import gensim.downloader as api
from transformers import pipeline

model=api.load("glove-twitter-25")

text="The forest is beautiful"

syn=[w for w,s in model.most_similar("beautiful",topn=3)]

new=text+" "+" ".join(syn)

gen=pipeline("text-generation")

print("\nOriginal:")
print(gen(text,max_length=30)[0]['generated_text'])

print("\nEnriched:")
print(gen(new,max_length=30)[0]['generated_text'])

```

5. Story Generation

```

import gensim.downloader as api
import random

model=api.load("glove-wiki-gigaword-50")

w=input("Enter word: ")

```

```
lst=[i for i,s in model.most_similar(w,topn=5)]
```

```
print("Story:")
```

```
print("A",w,"met",random.choice(lst),"near",random.choice(lst),"and found",random.choice(lst))
```

6. Sentiment Analysis

```
from transformers import pipeline
```

```
clf=pipeline("sentiment-analysis")
```

```
while True:
```

```
    t=input("Enter text (exit): ")
```

```
    if t=="exit":
```

```
        break
```

```
    r=clf(t)[0]
```

```
    print(r['label'],round(r['score'],2))
```

7. Summarization

```
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
```

```
name="sshleifer/distilbart-cnn-12-6"
```

```
tok=AutoTokenizer.from_pretrained(name)
```

```
mod=AutoModelForSeq2SeqLM.from_pretrained(name)
```

```
text=input("Enter text:\n")
```

```
inp=tok(text,return_tensors="pt")
```

```
out=mod.generate(inp["input_ids"],max_length=50)
```

```
print(tok.decode(out[0],skip_special_tokens=True))
```

8. LangChain

```
from transformers import pipeline
```

```
gen=pipeline("text-generation")
```

```
text=open("Artificial_Intelligence.txt").read()
```

```
out=gen("Summarize:\n"+text,max_length=50)
```

```
print(out[0]['generated_text'])
```

9. Wikipedia

```
from pydantic import BaseModel
import wikipedia
```

```
class A(BaseModel):
```

```
    name:str
    founder:str
    founded:str
    branches:str
    employees:str
    summary:str
```

```
n=input("Enter name: ")
```

```
p=wikipedia.page(n)
```

```
s=" ".join(p.summary.split(".")[4])+"."
```

```
d=A(name=p.title,founder="Check_Wiki",founded="Check_Wiki",branches="Many",employees="N/A",
summary=s)
```

```
print(d.json(indent=4))
```

10. IPC ChatBot

```
import PyPDF2
```

```
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
pdf=PyPDF2.PdfReader(open("ipc.pdf","rb"))
```

```
text=""
```

```
for p in pdf.pages[:20]:
```

```
    text+=p.extract_text()
```

```
sec=[i for i in text.split("Section") if len(i)>20]
```

```
while True:
```

```
    q=input("Ask (exit): ")
```

```
    if q=="exit":
```

```
        break
```

```
    vec=TfidfVectorizer()
```

```
    m=vec.fit_transform(sec+[q])
```

```
    s=cosine_similarity(m[-1],m[:-1]).flatten()
```

```
    print(sec[s.argmax()][:300])
```